# Shri Vaishnav Institute of Information Technology, Indore
# Shri Vaishnav Vidyapeeth Vishwavidyalaya

## Experiment : 01

**Aim :-** To study the definition, importance, and methodology of writing a Problem Statement in Object-Oriented Analysis and Design (OOAD).

**Theory :-**

**1. What is a Problem Statement?**

A problem statement is a clear, concise description of the issue that needs to be addressed by a problem-solving team. It focuses on the facts of the current situation and the gap between the current reality and the desired future state. It does not propose a solution or a method but clearly defines the "what" and the "why" of the project.

**2. Why is a Problem Statement Important?**

- **Focus**: It keeps the development team focused on the core issue and prevents them from getting distracted by unnecessary features.
- **Communication**: It serves as a communication tool between stakeholders and developers, ensuring everyone is on the same page.
- **Validation**: It allows the team to validate the solution against the original problem to ensure the objective is met.
- **Scope Management**: It helps in defining the boundaries of the project, preventing "scope creep."

**3. How to Write a Problem Statement (Methodology)**

The process of drafting a problem statement involves the following sequential steps:

**Step 1:** Study the Existing System Analyze the current process or system. Understand how things are done currently to identify inefficiencies, errors, or gaps.

**Step 2:** Identify the Problem Pinpoint the specific pain points or issues. Ask "What is wrong?" or "What is costing us time/money?"

**Step 3:** Define Objective State clearly what the system aims to achieve. The objective should be Specific, Measurable, Achievable, Relevant, and Time-bound (SMART).

**Step 4:** Define the Scope of the System Clearly define the boundaries.

*In-Scope:* What the project *will* cover.
*Out-of-Scope:* What the project *will not* cover.

**Step 5:** Identify Concepts Identify the key concepts and entities involved in the domain (e.g., in a Library system: Book, Member, Issue Date).

**4. Constraints (Constants)**

Every problem statement works within specific limitations. These are the "constants" that cannot be easily changed, such as:

- **Budget:** Financial limits.
- **Time:** Deadlines for delivery.
- **Technology:** Hardware or software restrictions.
- **Regulations:** Legal or compliance requirements.

## 5. Application Standard Format of Problem Statement

***1.*** **The Context**

Hotel operations like booking, billing and food orders are handled manually using registers. This makes record
management difficult and time-consuming.

***2.*** **The Problem**

Manual system causes billing errors, slow booking process, data loss and difficulty in tracking room availability
and customer details.

***3.*** **The Impact**

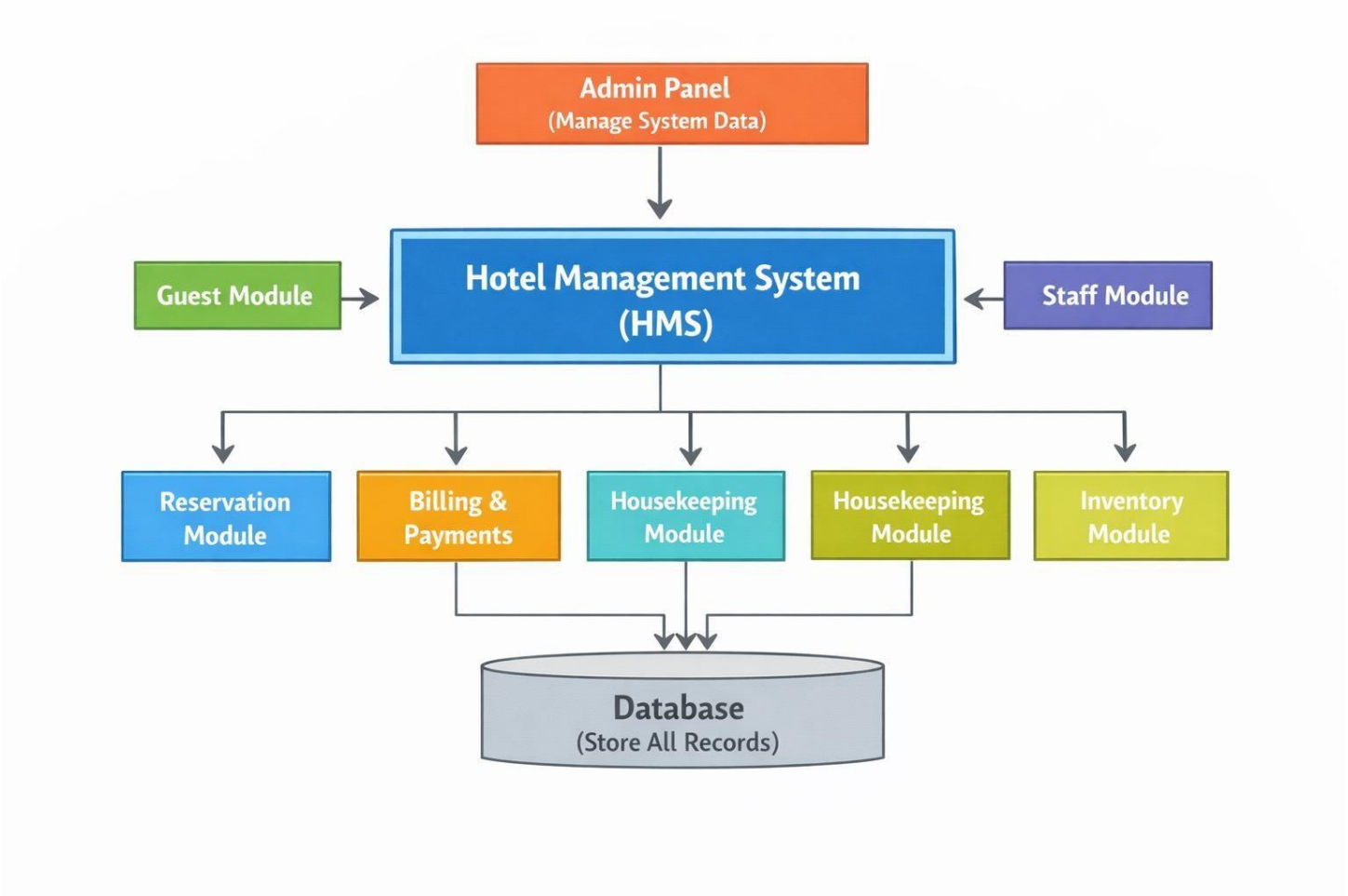
It leads to delays in service, poor management of records, financial loss and reduced customer satisfaction.

***4.*** **The Proposed Solution (Goal)**

A computerized Hotel Management System is proposed to automate booking, billing, customer records and food ordering to improve efficiency and accuracy.

## 6. Block Diagram

.



## 7. Conclusion

Writing a defined problem statement is the first and most critical step in software engineering. It ensures that the development team solves the *right* problem and provides a benchmark against which the final product can be measured.

## Experiment : 02

**Aim :-** Perform the system analysis: Requirements analysis, SRS

## Requirements :-

1. **Hardware Requirements**:

Hardware needs vary based on whether the system is hosted locally on a school server or accessed via the cloud.

• **Server Side (Centralized Host):**

Processor: Minimum Intel Core i5 or Intel Xeon for multi-user environments.
RAM: 8GB to 16GB minimum; 64GB+ is recommended for large institutions with hundreds of concurrent users.
Storage: 500GB to 1TB SSD (Solid State Drive) is preferred for faster database read/write speeds.
Network: A high-speed internet connection or a LAN connection using TCP/IP protocols for data transfer.

• **Client Side (User Devices):**

Processor: Intel Pentium 4 or higher (or equivalent modern mobile processor).
RAM: At least 2GB to 4GB of memory for smooth browser performance.
Display: Minimum screen resolution of 1024 x 768 for a desktop experience, though modern systems should be responsive for mobile screens.
Peripherals: Standard keyboard, mouse, and a printer (if physical report cards or receipts are needed).

2. **Software Requirements:**

This defines the technology stack used to build and run the application.

• **Operating System (OS):**

Server: Windows Server 2022 or Linux distributions like Ubuntu/CentOS.
Client: Any modern OS (Windows 10/11, macOS, Android, or iOS) capable of running a web browser.

**Database Management System (DBMS):**

Relational databases like MySQL, PostgreSQL, or Microsoft SQL Server (2019 or newer) are standard for structured student data.

• **Development Stack (Tech Stack):**

Backend: Java (J2EE), Python (Django), or C# (.NET).
Frontend: HTML5, CSS3, and JavaScript frameworks (like React or Angular).

• **Client Interface:**

A modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

**Theory:-**

A Software Requirement Specification (SRS) is a comprehensive document that describes what a software system should do and how it is expected to perform. It acts as a formal agreement between the customer and the development team, serving as the "single source of truth" for the entire project lifecycle.

A well-designed, well-written Software Requirement Specification (SRS) accomplishes four major goals that serve as the foundation for the entire project life cycle:

1. **Feedback to the Customer:** It provides the customer with assurance that the development organization has a clear and correct understanding of the problems to be solved and the software behaviour necessary to address them.
2. **Decomposition of the Problem:** The structured process of writing requirements helps break down complex problems into smaller, more manageable component parts in an orderly fashion.
3. **Input to Design Specifications:** The SRS acts as the "parent" document for subsequent project stages, providing sufficient detail on functional requirements so that an effective design solution can be devised.
4. **Product Validation Check:** It serves as the primary benchmark for testing and validation strategies, allowing the team to verify that the final product meets all agreed-upon requirements.

**SRS should address the following:**

The basic issues that the SRS shall Address are the following:-

- **Functionality:** It must explicitly state what the software is supposed to do, detailing every function and service the system provides.
- **External Interfaces:** It should define how the software interacts with people (user interfaces), hardware (device interfaces), and other software or communication protocols (APIs).
- **Performance:** It must specify static and dynamic requirements, such as response times, throughput, and the number of simultaneous users or transactions the system must support.
- **Attributes (Qualities):** It addresses non-functional "ilities," including **reliability, availability, security, maintainability,** and **portability.**
- **Design Constraints:** It must list any limitations imposed by the client or environment, such as required operating systems, hardware limitations, or compliance with specific standards and regulations.

**Characteristics of good SRS**

a. A good SRS document should have the following characteristics:

1. **Correct–** All requirements must accurately describe the client’s actual needs. There should be no wrong or unnecessary requirements.
2. **Complete–** The SRS should include all functional and non-functional requirements, constraints, assumptions, and system interfaces. Nothing important should be missing.

3. **Unambiguous (Clear)–** Each requirement should have only one meaning. It should be written in simple and clear language so that developers, testers, and clients understand it in the same way.

4. **Consistent–** There should be no conflicts between requirements. For example, one requirement should not say the system works offline while another says it works only online.
5. **Verifiable (Testable)–** Every requirement must be measurable and testable. It should be possible to check whether the system meets the requirement or not.
6. **Feasible–** Requirements should be practical and achievable within the given time, budget, and available technology.
7. **Traceable–** Each requirement should be traceable to its source (client, stakeholder, or document) and should be trackable throughout development and testing.
8. **Modifiable–** The document can be updated easily without major changes.

## A Sample of basic SRS Outline for Hotel Management System:

### 1. Introduction

#### 1.1 Purpose

The purpose of this document is to describe the functional and non-functional requirements of the Hotel Management System (HoMS). This document is intended for developers, project managers, and stakeholders.

#### 1.2 Scope

The Hotel Management System is designed to automate hotel operations such as room booking, customer management, billing, staff management, and report generation. The system will reduce manual work and improve efficiency.

#### 1.3 Abbreviations

HoMS – Hotel Management System
SRS – Software Requirements Specification
UI – User Interface
DB – Database

#### 1.4 References

IEEE SRS Standard
Hotel requirement documents
Project proposal document

### 2. Overall Description

#### 2.1 Product Perspective

The HoMS is a web-based/desktop-based system that maintains hotel records in a centralized database. It will be used by admin, receptionist, and hotel staff.

### 2.2 Product Functions

- Customer registration and record management
- Room management
- Room booking and reservation
- Billing and payment processing
- Staff management
- Inventory management (food, housekeeping items)
- Report generation

## 3. External Interface Requirements

### 3.1 User Interface

- Login page for authorized users
- Dashboard for admin and staff
- Customer registration form
- Room booking page
- Billing and report pages
- Simple and user-friendly design

### 3.2 Software Interface

- Database (MySQL or similar DBMS)
- Operating System (Windows/Linux)
- Web browser (Chrome, Edge)

1. **System Features**
   - Secure login and authentication
   - Add, update, delete customer records
   - Room availability checking
   - Room booking system
   - Automatic bill generation
   - Staff management
   - Inventory tracking
   - Report generation

## 5. Non-Functional Requirements

### 5.1 Performance

- System should respond within 2–3 seconds
- Should handle multiple users simultaneously

### 5.2 Safety

- Regular database backup
- Data recovery mechanism

### 5.3 Security

- Password-protected login
- Role-based access control
- Data confidentiality and encryption

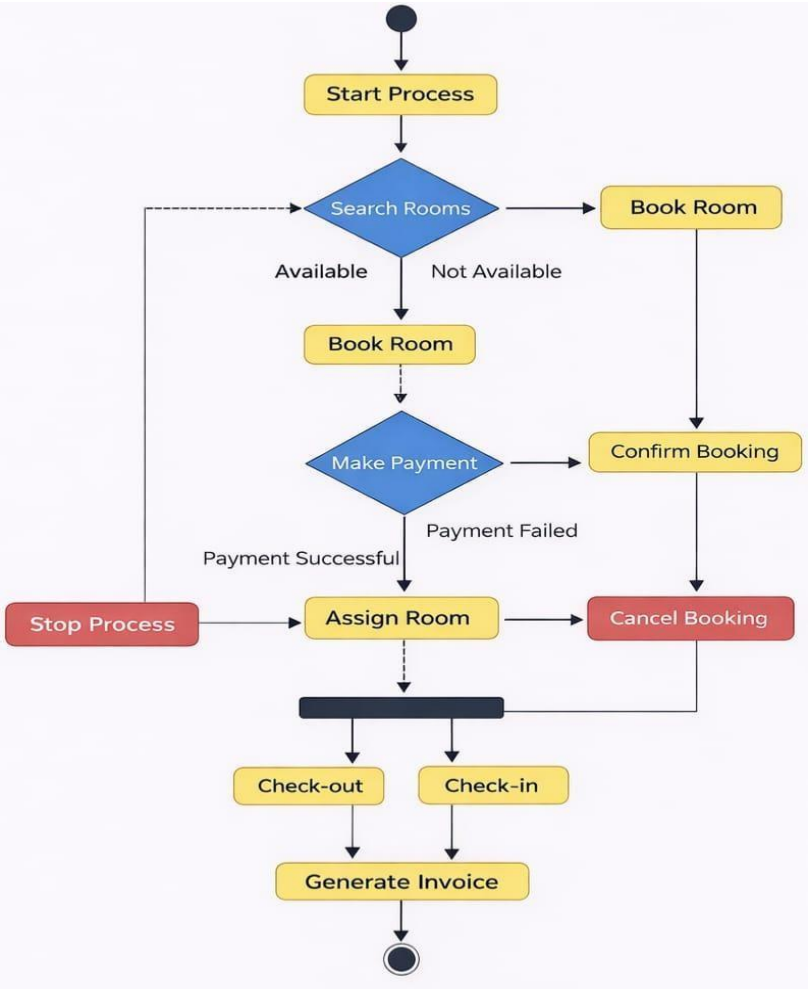
### 6. Other Requirements

### 6.1 User Characteristics

- Users should have basic computer knowledge.
- Admin should have system management knowledge.
- Staff should understand hotel operations.

## Result:-

## 1. Activity Diagram




### Conclusion

The Hotel Management System (HoMS) helps automate and manage hotel operations efficiently. It integrates modules like customer records, room booking, billing, staff, and inventory into one centralized system.

By reducing paperwork and improving accuracy, HoMS saves time, minimizes errors, and enhances overall service quality. It ensures secure data management and smooth coordination between departments, making hotel services faster and more organized.

# Experiment : 03

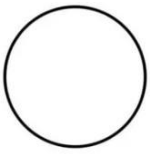
**Aim :-Study of DFD**

**Theory :-**

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system.
It shows the flow of data from input to processing and output.
DFD helps in understanding system functionality in a simple visual way.
It is widely used in system analysis and design.

**Components of DFD**:-

1. **Process** (Circle)
   Transforms input data into output
   (Example: Order Processing)



2. **Data Flow** (Arrow)
   Shows movement of data
   (Example: Customer Details)

3. **Data Store** (Open Rectangle)
   Stores data
   (Example: Database)

4. **External Entity** (Rectangle)
   Source or destination outside the system
   (Example: Customer)

**Rules for Creating DFD**:-

- Every process must have at least one input and one output
- No direct connection between **external entity and data store**
- Data flow must be properly labeled
- Process names should be **verb-based**
- Keep the diagram simple and clear

**Levels of DFD:-**

**Level 0:**

- Represents the entire system as a single process
- Shows only external entities and data flow

Example:
Customer → [System] → Admin

**Level 1:**

- Breaks the system into multiple processes
- Includes data stores

Example:
Customer → (Order Process) → Database
Database → (Billing Process) → Customer

**Level 2:**

- Further breaks Level 1 processes into detailed sub-processes

Example:
(Order Process) → Check Order → Validate Payment → Store Data

**Types of Data Flow Diagram (DFD)** :-

There are mainly **2 types of DFD**:

### 1. Logical DFD:

A Logical DFD shows **what the system does**.

### Key Points:

- Focuses on system functionality
- No technical details included
- Shows processes, data flow, and data stores

- Independent of technology

## 2. Physical DFD

A Physical DFD shows **how the system is implemented**.

### Key Points:

- Includes technical details
- Shows hardware, software, files, and people
- Explains actual implementation
- More detailed than logical DFD

**Advantages**:-

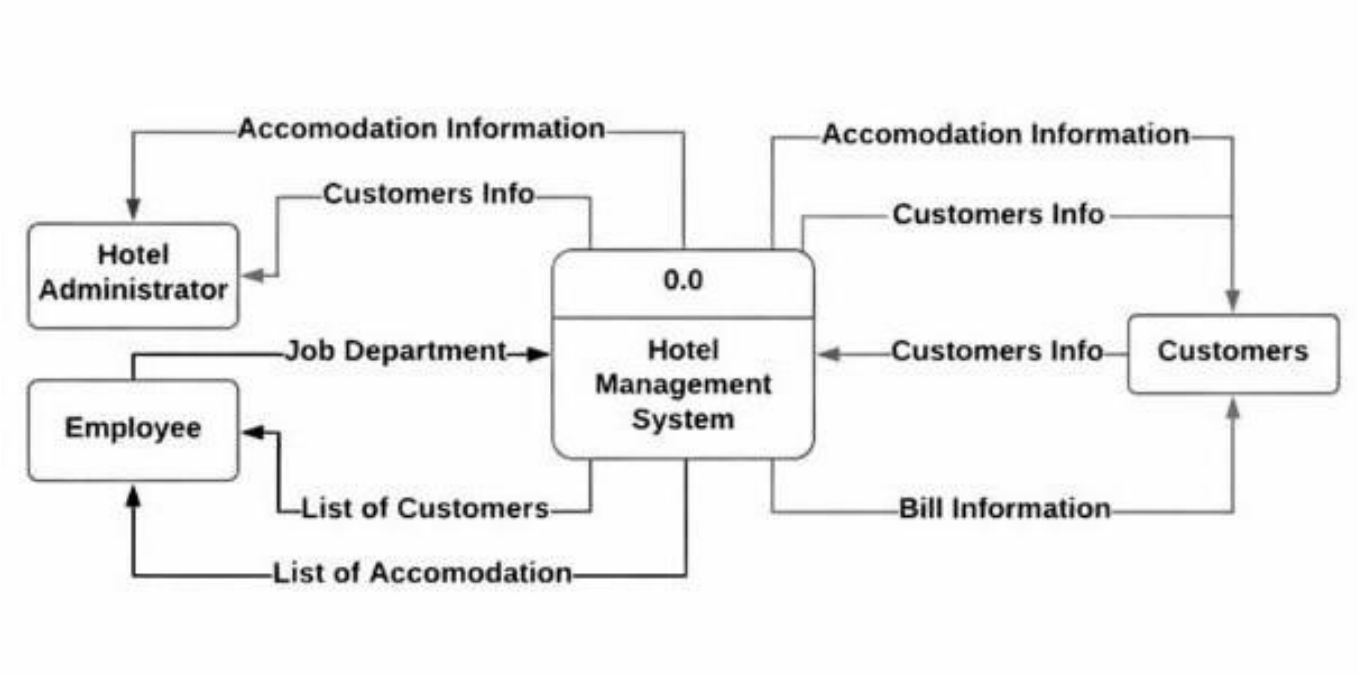
- Easy to understand and visualize
- Improves communication
- Helps in system analysis
- Detects errors easily

**Disadvantages**:-

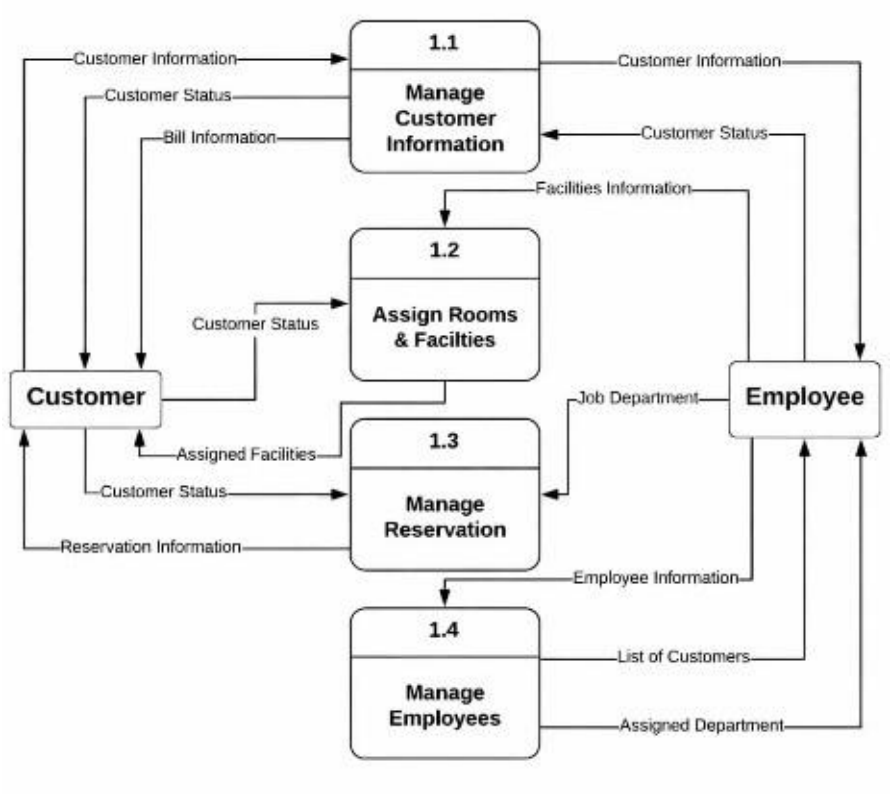
- Can become complex for large systems
- Time-consuming to create
- Does not show detailed implementation (in logical DFD)

**Result: :-**

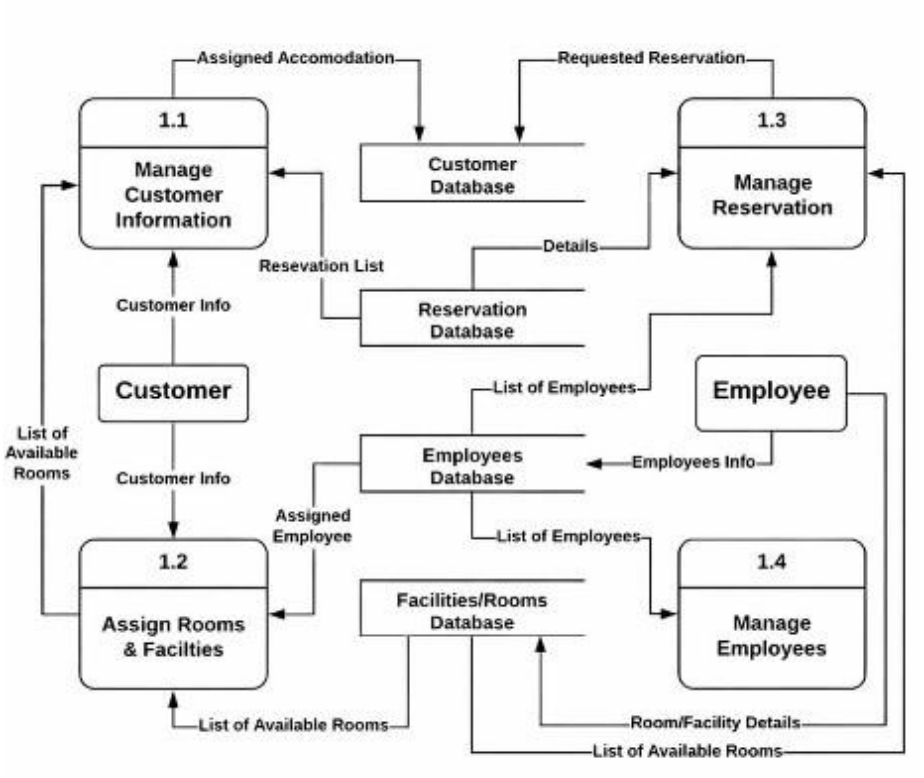
**Level 0:**




**Level 1:**


1.1
Manage Customer Information
1.2
Assign Rooms & Facilties
1.3
Manage Reservation
1.4
Manage Employees
Customer
Employee
Customer Information
Customer Status
Bill Information
Customer Information
Customer Status
Facilities Information
Customer Status
Assigned Facilities
Job Department
Customer Status
Reservation Information
Employee Information
List of Customers
Assigned Department

## Level 2:


Assigned Accomodation
Requested Reservation
1.1
Manage Customer Information
Customer Database
1.3
Manage Reservation
Details
Resevation List
Customer Info
Reservation Database
Customer
List of Employees
Employee
List of Available Rooms
Employees Database
Employees Info
Customer Info
Assigned Employee
List of Employees
1.2
Assign Rooms & Facilties
Facilities/Rooms Database
1.4
Manage Employees
List of Available Rooms
Room/Facility Details
List of Available Rooms

# Shri Vaishnav Institute of Information Technology, Indore
# Shri Vaishnav Vidyapeeth Vishwavidyalaya

## Experiment : 04

## Aim:

## 1. Introduction of Use Case:

A **Use Case Diagram** is a type of UML (Unified Modeling Language) diagram used to show **how users (actors) interact with a system**.

**It represents:**

- What the system does (functions)
- Who uses the system (actors)
- Interaction between user and system

## 2. Elements of Use Case Diagram with Notation:

| Element | Meaning | Notation |
|---|---|---|
| **Actor** | User or external system | (stick figure) |
| **Use Case** | Function of system | ○ (oval shape) |
| **System Boundary** | System limit | ⬭ (rectangle box) |
| **Association** | Interaction line | —— (line) |
| **Include** | Mandatory relation | → <<include>> |
| **Extend** | Optional relation | → <<extend>> |
| **Generalization** | Inheritance | → (arrow line) |

## 3. Characteristics of Use Case:

- Focus on **user requirements**
- ✓ Shows **external view** of system (not internal logic)
- ✓ Easy to understand
- ✓ Helps in **communication between users & developers**
- ✓ Describes **functional requirements only**
- ✓ Independent of implementation

## 4. How to Write Use Case:

- **Identify Actors** :

Who will use the system (User, Admin)

- **Identify Use Cases** :

What tasks the system will perform (Login, Register)

- **Define Relationships** :
  - **<<include>> (Mandatory):**
    One use case always includes another use case.
    Example: Login is included in Shopping
  - **<<extend>> (Optional):**
    Adds extra or optional behavior to a use case.
    Example: Discount extends Payment

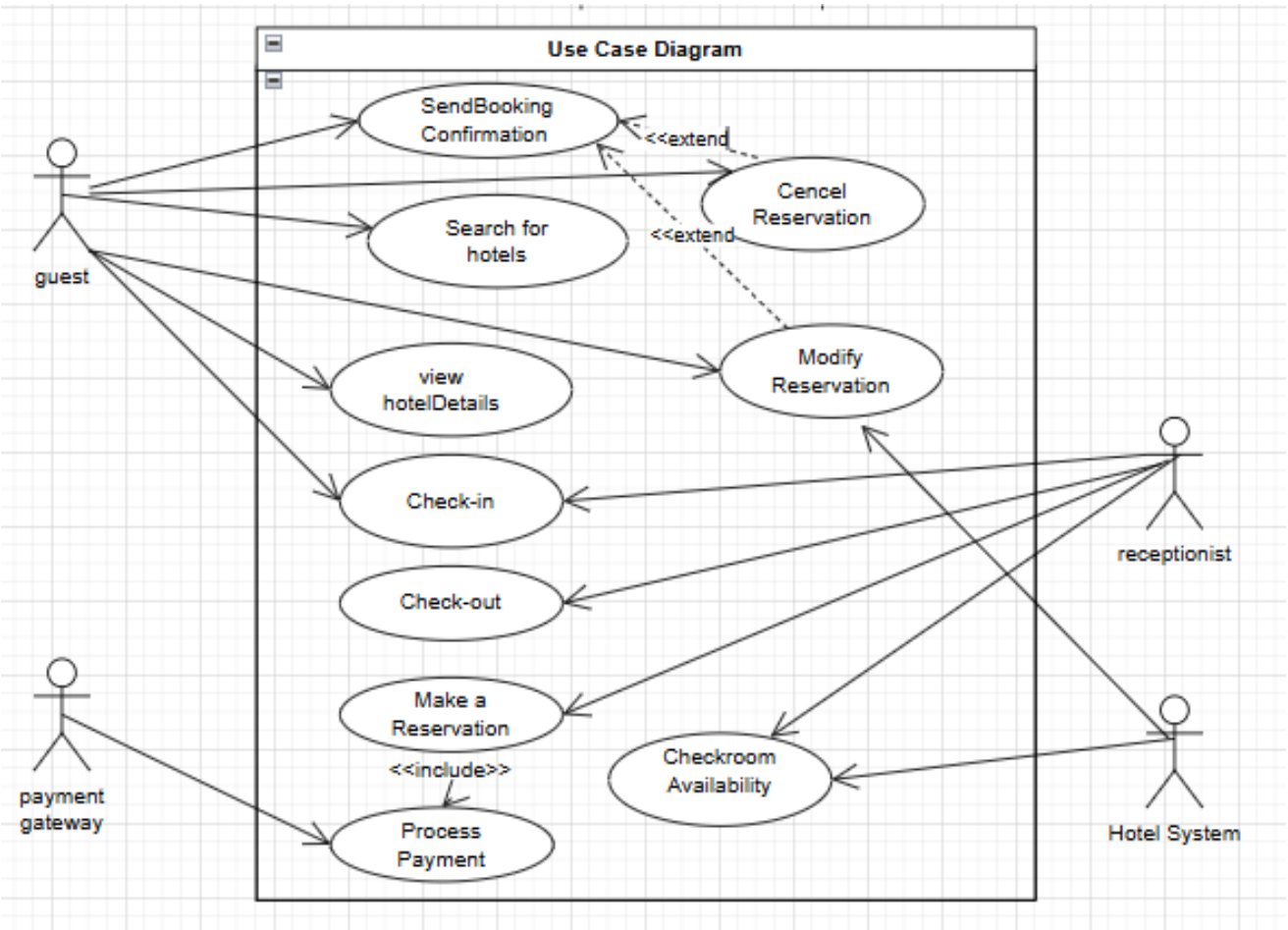
- **Draw Diagram** :Actor , Use Case , Lines connect karo

- **Write Description** :Name, Actor, Pre-condition, Flow, Post-condition

## 5. Benefits of Use Case:

- Easy to understand for non-technical people
- Helps in requirement gathering
- Improves system design
- Identifies system functionality clearly
- Helps in testing (test cases)
- Reduces confusion between team members

## 6. Result:



## 7. Conclusion:

A use case clearly describes how users interact with a system to achieve specific goals.
It helps in understanding system functionality in a simple and structured way.
By defining actors, use cases, and relationships like <<include>> and <<extend>>, it improves clarity and design.

# Experiment : 05

## 1. Introduction of Class Diagram:

A **Class Diagram** is an important diagram in **UML (Unified Modeling Language)** that represents the **static structure** of a system.

It shows:

- the **classes** in the system
- their **attributes (data members)**
- and their **methods (functions)**

It also represents the **relationships** between classes, such as association, inheritance, and dependency.

## 2. Purpose of Class Diagram:

- System ka **design samajhne ke liye**
- Developers ko **coding me help**
- Object-oriented system ko **visual form me dikhane ke liye**
- Classes ke beech **relationship samajhne ke liye**

### 1. Class

A class is the main building block of the diagram.
It is represented by a **rectangle divided into three parts**:

- Class Name
- Attributes
- Methods

### 2. Attributes

These are the **properties or variables** of a class.
They describe the **data** stored in the class.

Example:

- name
- age
- id

### 3. Methods (Operations)

These are the **functions or behaviors** of a class.
They define what the class can do.

Example:

- display()
- calculate()

### 5. Visibility (Access Specifiers)

☞ Shows access level of attributes and methods:

- Public
- Private
- Protected

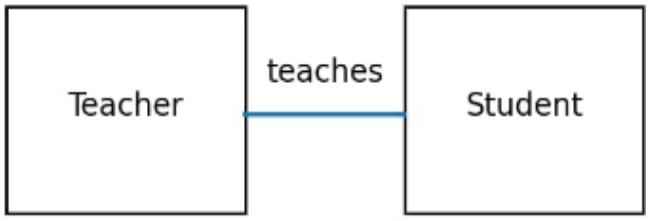
Example:

```
+ name
- salary
# age
```

## 3. Relationships:

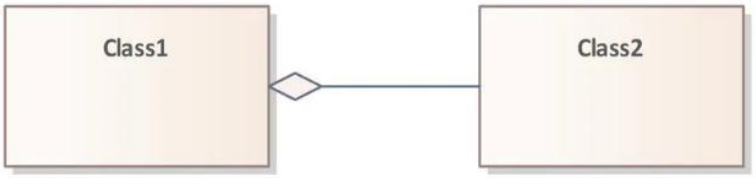
### 1. Association

A **basic relationship** between two classes
Shows that objects of one class are connected to another



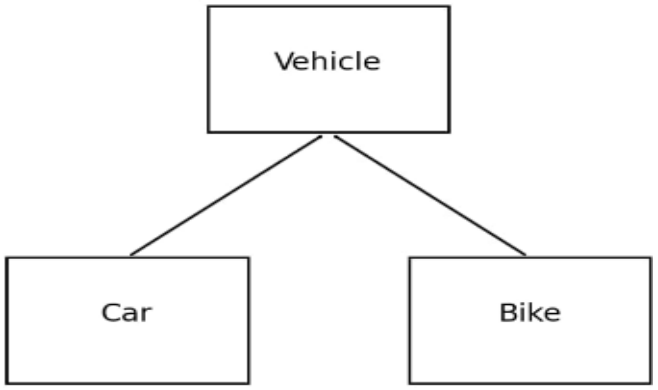

### 2. Aggregation

A **whole-part relationship**, but objects can exist independently



## 4. Generalization

Shows **inheritance** (one class inherits another)

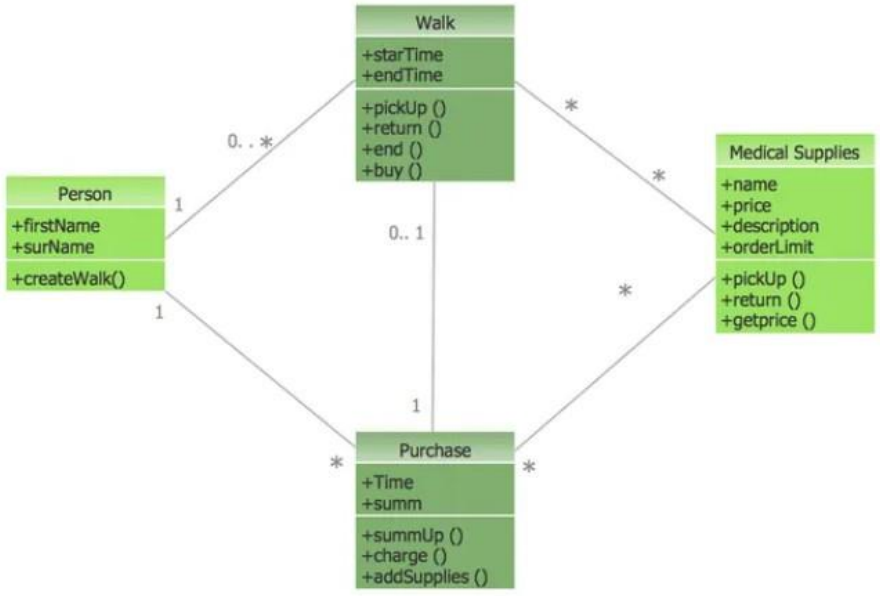



## 5. Dependency

One class **depends on another temporarily**


## 6. Multiplicity

Shows **how many objects** are related



## Metadata:

**Metadata** refers to **"data about data"**. In the context of class diagrams or software systems, metadata provides **additional information or description** about data, attributes, or classes.

It does not represent actual data, but gives **details about the data**.

Example:

- Data: `name = "Rahul"`
- Metadata:
  - Data type → String
  - Length → 50
  - Required → Yes

## Abstract Class:

An **Abstract Class** is a class that **cannot be instantiated (object nahi ban sakta)** and is used as a **base class** for other classes.

It may contain:

- Abstract methods (no implementation)
- Normal methods (with implementation)

Other classes **inherit** from it and implement its abstract methods.

## Object Diagram:

An **Object Diagram** is a UML diagram that represents the **instance (real objects)** of classes at a particular moment in time. It is basically a **snapshot of the system** showing how objects are created and how they are connected with each other.

While a class diagram shows the **structure (blueprint)**, an object diagram shows the **actual implementation (real example)** of that structure.

It includes:

- Objects (instances of classes)
- Attribute values
- Links (connections between objects)

Example:

If class is **Student**, then object diagram will show:

```
student1 : Student
name = "Rahul"
roll_no = 101
```

## Difference Between Class Diagram and Object Diagram:

| Basis | Class Diagram | Object Diagram |
|---|---|---|
| **Definition** | Shows the **structure of the system** with classes | Shows the **actual objects (instances)** of classes |
| **Nature** | **Static (design level)** | **Snapshot (runtime view)** |
| **Focus** | Classes, attributes, methods | Objects and their **values** |
| **Representation** | Blueprint of system | Real example of system |
| **Time** | Not time-specific | Shows state at a **particular moment** |
| **Details** | General structure | Specific data (actual values) |
| **Example** | `Student` class | `student1 : Student` |

## Result: